SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Nandhakumaaran D
192511312RD

Application Based Questions

← Back

QUESTIONS

- Q1 Student Grade Book Application 10 marks ✓
- Q2 Debugging and Rewriting 10 marks ✓

2/2 answered

Q1 Student Grade Book Application

10 marks

You are building a student grade book application. Write a Python program that:
(a) Stores each student's name and marks in a dictionary (key = name, value = list of 5 subject marks).
(b) Computes total, average, and grade (O: ≥90, A+: ≥80, A: ≥70, B+: ≥60, B: ≥50, F: <50) for each student.
(c) Finds the class topper (highest average) and the subject with the highest overall mean.
(d) Displays a formatted result table sorted by average in descending order.
Use dictionary comprehension and list operations. Minimum 5 students.
Write complete, syntactically correct Python programs using lists, tuples, dictionaries, and sets with proper indentation.

Your Code

```python
students = {
    "Arun": [85, 90, 78, 92, 88],
    "Bala": [60, 55, 70, 65, 50],
    "Charan": [95, 92, 88, 91, 94],
    "Divya": [45, 50, 40, 55, 48],
    "Esha": [75, 80, 70, 78, 82]
}

def get_grade(avg):
    if avg >= 90:
        return "O"
    elif avg >= 80:
        return "A+"
    elif avg >= 70:
        return "A"
    elif avg >= 60:
        return "B+"
    elif avg >= 50:
        return "B"
    else:
        return "F"

student_data = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": sum(marks) / len(marks),
        "grade": get_grade(sum(marks) / len(marks))
    }
    for name, marks in students.items()
}

topper = max(student_data, key=lambda name: student_data[name]["averag
subject_names = ["Sub1", "Sub2", "Sub3", "Sub4", "Sub5"]
subject_means = [
    sum(students[name][i] for name in students) / len(students)
    for i in range(5)
]
best_subject_index = subject_means.index(max(subject_means))
best_subject = subject_names[best_subject_index]

sorted_students = sorted(
    student_data.items(),
```

Input

stdin input

Output

```
Name      Total     Average
Grade
---------------------------------
---------
Charan    460       92.00     O
Arun      433       86.60
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

PYTHON PROGRAMMING

Nandhakumaaran D
192511312RD

Application Based Questions

← Back

QUESTIONS

Q1
Student Grade Book Application
10 marks

Q2
Debugging and Rewriting
10 marks

2/2 answered

Q2 Debugging and Rewriting

10 marks

The program below simulates a simple banking system using lists and dictionaries. It has FOUR bugs. Identify each bug with reason and rewrite the correct program.

```
accounts = {}
def create_account(acc_no, name, balance):
accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
def deposit(acc_no, amount):
if acc_no in accounts:
accounts[acc_no]['balance'] =+ amount
accounts[acc_no]['txns'].append(('Deposit', amount))
def withdraw(acc_no, amount):
if accounts[acc_no]['balance'] > amount:
accounts[acc_no]['balance'] -= amount
accounts[acc_no]['txns'].append(('Withdraw', amount))
else:
print('Insufficient balance')
def mini_statement(acc_no):
for t in accounts[acc_no]['txns'][-5]:
print(t)
create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

For debugging questions: clearly state the error, the line number, the reason, and the corrected line.

Your Code

```python
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))
    else:
        print('Account not found')

def withdraw(acc_no, amount):
    if acc_no in accounts and accounts[acc_no]['balance'] >= amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print('Insufficient balance')

def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)

```

Input

```
create_account(1001, 'Ravi',
10000)
deposit(1001, 5000)
withdraw(1001, 20000)
```

Output

```
Insufficient balance
('Deposit', 5000)
('Withdraw', 3000)
Balance: 12000
```